TECHNICAL REPORT · CAPSTONE 2

AI-Powered Phishing Awareness & Risk-Scoring Platform

A human-risk measurement and remediation platform built around a measurement/delivery boundary

Team Sanad Al-Khatib · Omar Al Sheikh Khalil · Ghassan Hittini · Saad Thibian

Supervisors Dr. Mariam Biltawi · Eng. Sami Al-Mashaqbeh

Institution Al-Hussein Technical University · B.Sc. Cyber Security & Computer Science

Build window Oct 2026 – Feb 2027

Contents

	Abstract
1	Introduction — motivation, problem, objectives, scope, scope evolution, contributions
2	Background and Related Work
3	Requirements
4	System Architecture
5	Data Model
6	The Scoring Model
7	Ingestion and Integration
8	Directory Integration (Entra ID & Active Directory)
9	GoPhish Campaign Control
10	The Awareness & Remediation Portal
11	API Design
12	Security and Privacy Engineering
13	The Measurement/Delivery Boundary
14	Implementation
15	Testing and Verification
16	Evaluation
17	Results and Discussion
18	Limitations
19	Future Work
20	Conclusion
	References
A	Appendix A — Lure Difficulty Tiers
B	Appendix B — Running the Platform
C	Appendix C — Selected Constants
D	Appendix D — Security Controls at a Glance
E	Appendix E — API Surface

Abstract

Phishing remains the dominant initial-access vector in real-world intrusions, and its decisive variable is human behaviour rather than any single perimeter control [8][9][10]. Security-awareness programmes attempt to manage this human risk by running simulated phishing campaigns, but most reduce the outcome to a single organisation-wide click rate — a scalar that discards who is at risk, how deeply they were compromised, how difficult the lure was, and whether behaviour is improving over time.

This report presents a complete phishing human-risk platform built around a firm ethical boundary. At its heart is a **measurement and analytics layer**: a multi-tenant web platform that ingests interaction-event metadata from authorised internal simulations and transforms it into per-employee Human Risk Scores (HRS), a Susceptibility Index, an Org-Risk prioritisation overlay, departmental heat-maps, campaign funnels, and decayed time-series trends. The scoring model is an explicit three-layer function — compromise depth, tier-difficulty weighting, and decayed aggregation — implemented as pure, unit-tested functions whose parameters are centralised for calibration.

Around that core the platform adds the capabilities an organisation needs to run the whole programme: enterprise directory sync (Microsoft Entra ID and on-prem Active Directory over LDAP) to populate the roster; full GoPhish campaign control from one console under an authorisation gate and audit trail; and a separate Awareness & Remediation Portal that closes the loop — turning a high score into mandatory, non-skippable training and, for people who stay elevated too long, an automatic in-person-awareness mark plus an outbound ticket to the organisation's ticketing system.

A defining architectural constraint holds throughout: **the platform measures attacks and trains people, but never manufactures attacks.** It does not generate lures, clone landing pages, capture credentials, or send email; the deception artefacts stay authored inside GoPhish [1], a separate MIT-licensed tool the team operates under its own authorisation. By construction there is no credential column in the schema, ingestion discards any submitted credential payload, and the console orchestrates GoPhish without authoring its lures. Section 1.5 explains why the generative component proposed in Capstone 1 was deliberately not built, and Section 13 argues that this separation is a design strength rather than a shortfall.

The system is implemented as a FastAPI/SQLAlchemy backend and a React/Vite console, with the remediation portal as an independent FastAPI service and single-page app. It was reviewed and hardened under a dedicated secure-coding pass covering security headers and CSP, JWT-expiry enforcement, cross-tenant integrity guards, upload caps, HTML sanitisation, SSRF scheme checks, fail-closed-in-production ingestion and rate limiting, followed by a second pass adding **encryption at rest for every integration secret, httpOnly-cookie sessions with double-submit CSRF protection, and least-privilege per-tenant service tokens** for cross-service automation. It is covered by **119 automated tests** plus a nine-scenario scoring-sanity gate, passes clean dependency audits, and was demonstrated end-to-end against a live GoPhish instance (a real open→click→submit chain scored to HRS 61.1, "High", with credentials dropped on arrival) and across both services.

1. Introduction

1.1 Motivation

A click rate answers one question — "how many people clicked?" — and hides everything a security or GRC team must act on. Two organisations with the same 12 % click rate can carry entirely different risk: one where a handful of executives submitted credentials to a targeted spear-phish, and one where a broad set of staff clicked an obvious bulk lure but none proceeded. Effective human-risk management needs a per-person, depth-aware, difficulty-aware and time-aware measure — and then it needs to act on that measure. Producing the measure from a stream of simulation events, and closing the loop from measurement to remediation, is the problem this project solves.

1.2 Problem statement

Given a stream of interaction events emitted by authorised phishing simulations — `opened`, `clicked`, landing-page `dwell`, credential `field_focus`, `typed_no_submit`, `submitted`, attachment `download`, `chatbot_turn`, `chatbot_disclosure` and `reported` — compute for each employee a stable, explainable, banded risk score that reflects (a) how deeply they were compromised, (b) how difficult the lure was, and (c) how recently it occurred, while never handling or persisting any sensitive credential value; then drive an automatic remediation response from that score, and let an operator run the whole programme — roster, campaigns and follow-up — from one place.

1.3 Objectives

1. Define a transparent, tunable scoring model mapping behavioural evidence to a 0–100 banded HRS and a complementary Susceptibility Index.
2. Provide role-scoped dashboards, drill-downs and decayed trends for programme owners, operators and departmental viewers.
3. Integrate with the team's existing simulator (GoPhish) so real campaign results flow into scoring automatically, and let an operator run the full campaign lifecycle from one console.
4. Populate and maintain the roster automatically from the organisation's directory (Entra ID or on-prem Active Directory).
5. Close the loop with a remediation portal that turns risk into mandatory training and, for persistent risk, an escalation to in-person awareness and a ticket.
6. Enforce a strict data-handling boundary — metadata only, no credential capture or storage, no lure authoring in the scorer — as a construction property, not a policy.
7. Deliver tested, security-reviewed, runnable software with documentation adequate for a capstone defence.

1.4 Scope and non-goals

The measurement core is exclusively the measurement layer. The following remain out of scope by design and stay inside the delivery tool (GoPhish): lure and email generation, landing-page cloning, credential capture or storage, and email sending. The console can orchestrate GoPhish — build groups and launch campaigns that reference templates and landing pages authored in GoPhish — but it never authors or clones those deception artefacts itself, and the remediation portal is strictly defensive.

1.5 From Capstone 1 to Capstone 2: what changed, and why

Capstone 1 specified four core services: a Contextual Profile Database, a **Generative AI Engine** that would turn organisational context into tiered spear-phishing emails and chatbot dialogue, a Simulation Web Application hosting cloned portals and an IT-support chatbot, and an Analytics & Risk-Scoring Engine. Capstone 2 delivers the first and fourth in full, delivers the third only in its non-deceptive form (a metadata beacon on a GoPhish-hosted page), and **deliberately does not build the second at all.** This is the most significant divergence between the proposal and the delivered system, and it was a considered decision rather than a shortfall of time.

The reasoning is threefold. **First, the artefact itself.** A component that ingests a real organisation's directory data and emits personalised, context-aware spear-phishing text is — in function, not merely in appearance — an offensive capability. Recent work demonstrates exactly this: generative models can already produce context-aware spear phishing against named individuals from public social-media data [15], and LLM agents are being studied on both sides of the phishing problem [14]. Building another generator would have added a real-world hazard to the project while contributing nothing to the research question the platform actually answers.

Second, the measurement question does not require it. The scientific contribution here is the scoring model: how behavioural evidence, lure difficulty and recency combine into a defensible per-person risk measure. That model is validated by the events that come back, and it is entirely indifferent to whether the lure that produced them was written by a human, a template or a model. GoPhish already authors and sends lures under the operator's own authorisation, so generating them a second time inside the scorer would duplicate an existing capability while collapsing the boundary that makes the scorer safe to run.

Third, the ethics of synthesis are an active research area, not a settled one. Work on ethically grounded LLM-based threat synthesis stresses containment, consent and governance as preconditions [16] — preconditions a student project running against a university's own staff could not credibly guarantee. Declining to build the generator was the position most consistent with that literature.

> **On the project title.** The platform is named "AI-Powered", and it is right to be explicit that the delivered system contains no machine-learning model or LLM: the scoring engine is deterministic, inspectable arithmetic, and that is a deliberate property (Section 6.10) — a score that informs people-facing decisions must be explainable and re-tunable, which an opaque model would not be. The "AI" ambition of Capstone 1 survives as the roadmap in Section 19, where the defensible applications are adaptive difficulty selection and narrative explanation of scores, not lure generation. A reader comparing the two documents should read the change as a scope decision taken on safety grounds, made early and carried consistently through the architecture.

1.6 Contributions

- An explicit, inspectable three-layer human-risk scoring model with an inverted severity/susceptibility formulation (Section 6).
- A single-console integration pattern that unifies the whole campaign lifecycle while preserving a hard measurement/delivery boundary (Sections 9, 13).
- A closed remediation loop: an independent training service that consumes scores over REST and writes back a baseline, an in-person mark and a ticket — without ever crossing the measurement core's boundary (Section 10).
- Enterprise directory sync from Entra ID and on-prem AD (Section 8), and a construction-level "no credential surface" guarantee enforced at the schema, ingestion and client-instrumentation layers (Section 12.2).
- A worked example of security engineering as an iterative discipline: a documented two-pass hardening programme that moved secrets to encryption at rest, sessions to httpOnly cookies with CSRF protection, and cross-service automation to least-privilege tokens (Section 12).

2. Background and Related Work

2.1 The problem domain

Phishing is consistently reported as the most common entry point for organisational compromise, and the associated breach costs remain among the highest of any initial-access vector [8][10]. Industry telemetry shows sustained attack volume with a continuing shift toward targeted, credential-oriented campaigns [9]. Because the exploited component is a person rather than a system, the residual risk cannot be closed by perimeter controls alone, which is what motivates organisation-run awareness programmes.

2.2 Automated phishing detection — a different problem

A large body of recent work addresses the *detection* problem: classifying whether a given email or URL is malicious. Comparative studies of deep-learning models for phishing email detection report strong benchmark performance [11], ensemble approaches such as PhishGuard target website detection specifically [12], and multimodal systems now combine page content and structure with explainability techniques so an analyst can see why a verdict was reached [13]. More recently, LLM-based multi-agent systems have been applied to the same classification task [14].

This project deliberately addresses a **different** question. Detection asks "is this message malicious?"; this platform asks "given that a benign simulated message was delivered, how did this person behave, and what does that imply about their risk?" The two are complementary: detection reduces how many lures reach a person, and human-risk measurement addresses the residual that gets through. Nothing in the detection literature produces a per-person, longitudinal, decision-ready risk measure, which is the gap this work targets.

2.3 Behavioural analytics and human-risk scoring

The methodological lineage for the scoring model is User and Entity Behaviour Analytics, where per-entity baselines and weighted behavioural signals are aggregated into a risk score [17]. Related insider-threat work applies similar behavioural aggregation to a different threat population and surveys the modelling trade-offs involved [18][19]. Mature awareness programmes generalise the click rate along the same lines, weighting *depth* (a credential submission is worse than a click), *difficulty* (falling for a targeted lure differs from falling for a crude one) and *recency* (old failures should fade as behaviour improves).

This project operationalises those three principles as an explicit function rather than an opaque score, so that every number is defensible and re-tunable — a requirement in a GRC context where scores inform people-facing decisions. Where much of the UEBA literature pursues predictive accuracy through learned models, the design choice here is the opposite: full transparency, validated by property-based testing (Section 16).

2.4 Commercial awareness platforms

Commercial human-risk platforms — KnowBe4 [22], Proofpoint [23] and Hoxhunt [24] — combine simulation delivery, training content and reporting in a single hosted product, and the "human risk management" framing is now explicit in vendor guidance [21]. Practitioner analyses of awareness-programme failure modes highlight recurring problems: engagement decay, training that is not targeted to measured risk, and metrics that do not survive scrutiny [20].

Two differences define this project's position. First, those platforms are closed: the scoring function is not inspectable, so an organisation cannot audit or re-tune why a given employee scored as they did. Second, they bundle delivery and measurement in one system. This project separates them on purpose (Section 13), which costs integration convenience but yields an auditable system of record that never touches the deceptive act it measures.

2.5 Simulation delivery, training standards and identity

GoPhish [1] is a widely used open-source (MIT) framework that composes and sends simulation email, hosts tracked landing pages, and exposes results via a REST API and outbound webhooks; each recipient is issued a tracking id and events are recorded as a per-campaign timeline. **Training content and standards:** standards-aligned awareness material is freely available and openly reusable [6], and interoperability standards such as SCORM and xAPI define how training completion is tracked [5]; the remediation portal follows the same model — non-skippable lessons, a passed exam, completion tracking — while referencing free content an organisation can extend. **Security control frameworks:** NIST SP 800-50 and SP 800-53 [4] situate awareness training within a control catalogue, and OWASP ASVS [3] provided the checklist against which the application itself was reviewed. **Directory integration:** enterprises manage identity in Entra ID, read over the Microsoft Graph REST API under an app registration with an application permission [2], or in on-prem Active Directory over LDAP; both expose the person metadata needed to seed a roster, and neither requires reading anyone's password.

3. Requirements

3.1 Functional requirements

- **FR1 — Ingestion.** Accept interaction-event metadata via JSON, CSV, an HMAC-signed GoPhish webhook, a landing-page beacon, and a read-only GoPhish pull. Reject credential-shaped fields.
- **FR2 — Scoring.** Compute per-campaign compromise depth, per-employee HRS, Susceptibility Index, penetration ratio and Org-Risk, with configurable aggregation.
- **FR3 — Dashboards.** Role-scoped Overview, People (ranked, filterable, sortable), per-person drill-down, and decayed Trends.
- **FR4 — Management.** CRUD for campaigns, roster and console users; CSV import/export; self-service integration configuration with secret rotation.
- **FR5 — Multi-tenancy.** Every operational record is tenant-scoped; a departmental viewer sees only its department.
- **FR6 — Directory sync.** Import and refresh the roster from Entra ID or on-prem Active Directory, reading directory metadata only.
- **FR7 — GoPhish control.** From the console, build target groups from the roster and create, launch, schedule, complete or delete campaigns that reference GoPhish-authored content — under an explicit authorisation confirmation and full audit.
- **FR8 — Remediation.** A separate portal delivers non-skippable training and exams, publishes policies for acknowledgement, and automatically assigns remediation on failure or on a new policy; onboarding pushes a Moderate baseline into scoring, and persistent High/Critical risk raises an in-person-awareness mark and an outbound ticket.

3.2 Non-functional requirements

- **NFR1 — Safety.** No credential value is accepted or stored anywhere (construction-level); the console never authors or clones a lure or landing page.
- **NFR2 — Security.** Authenticated sessions with expiry enforcement, RBAC, per-tenant HMAC secrets, encryption at rest for integration secrets, CSRF protection for cookie-authenticated mutations, security response headers and CSP, locked CORS, production secret fail-fast, per-IP rate limiting, upload caps, server-side HTML sanitisation, SSRF scheme validation, and audit logging (Section 12).
- **NFR3 — Determinism and testability.** The scoring engine is pure (no I/O) and unit-tested; identical inputs yield identical scores.
- **NFR4 — Operability.** Zero-config local run (SQLite default); containerised Postgres for deployment; a single-command launcher starts all services.
- **NFR5 — Auditability and observability.** Side-effectful actions are recorded; scores are reproducible from stored events by a recompute; misconfiguration that is deliberately not exposed in a public API response is still recoverable from server-side logs and the audit trail (Section 12.7).

4. System Architecture

4.1 Layered overview

The platform is organised around a boundary-clean measurement core with three families of capability attached to it: **inbound integration** (how simulation events arrive), **operator control** (directory sync and GoPhish

orchestration), and a **remediation loop** (a separate portal that consumes scores and drives training and escalation). GoPhish remains the delivery layer — it, and only it, authors lures and landing pages and sends mail. The two application services each own their own database and talk to each other only over a narrow REST contract.

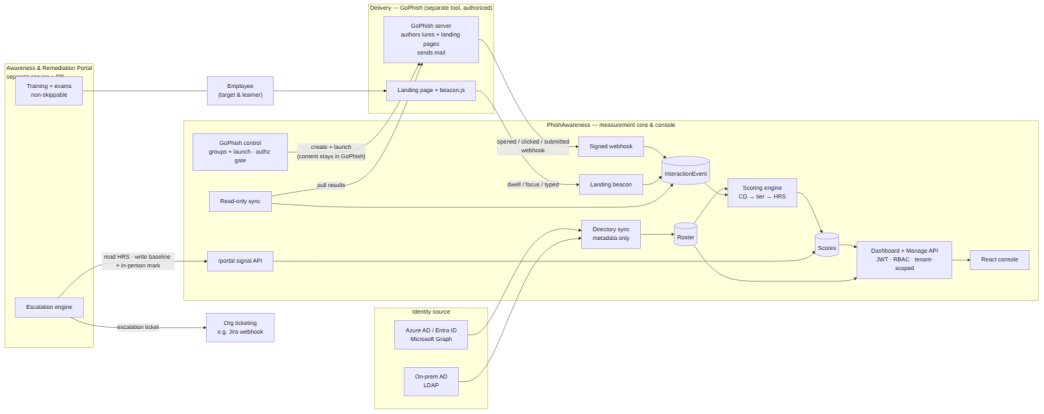


Figure 4.1 — System architecture. GoPhish delivers; the measurement core ingests event metadata (signed webhook, landing beacon, read-only sync) and serves one console; operator capabilities add directory sync and full GoPhish campaign control; a separate portal consumes scores over REST and pushes back a baseline floor and an in-person mark.

4.2 Components

- **Scoring engine** (`backend/app/scoring/`): pure functions (`score_campaign` , `score_employee`) plus a centralised `constants.py` ; no database or network access, enabling exhaustive unit testing.
- **Scoring service** (`scoring/service.py`): binds the engine to persistence — `score_tenant_from_db` reads `InteractionEvent` rows, resolves per-tenant settings and applies any baseline floor written by the portal.
- **Ingestion and dashboard API** (`api/`): event ingestion, five role-scoped dashboard views and trend endpoints.
- **Management and integration API** (`api/routes_manage.py`): campaign/roster/user CRUD, CSV import/export, scoring settings, GoPhish sync and full campaign control, and directory sync.
- **Portal signal API** (`api/routes_portal.py`): the REST surface the remediation portal calls to read scores and write a baseline floor and an in-person-awareness mark, authenticated by a scoped service token (Section 12.6).
- **Integrations** (`app/integrations/`): `gophish_webhook.py` , `beacon.py` , `gophish_sync.py` , `gophish_admin.py` , `entra.py` , `ldap_dir.py` , and the shared `directory_sync.py` upsert used by both directory connectors.
- **Security and platform** (`api/security.py` , `api/csrf.py` , `security_headers.py` , `ratelimit.py` , `uploads.py` , `crypto.py` , `logging_config.py`): session and password handling, CSRF verification, response-header/CSP middleware, rate limiting, capped uploads, encryption at rest, and application logging.
- **Console** (`frontend/`): React/Vite — Overview, People, Trends, Manage, Integration — with hand-rolled SVG charts.
- **Awareness & Remediation Portal** (`portal/`): an independent FastAPI service and single-page app with its own SQLite database, employee logins, training modules and exams, policies, an escalation engine and an outbound ticket integration.

4.3 Technology choices and rationale

Concern	Choice	Rationale
API framework	FastAPI	Typed request/response via Pydantic; automatic OpenAPI; async-ready.
ORM	SQLAlchemy 2.0	Typed models; SQLite↔Postgres portability; `TypeDecorator` enabled transparent encryption at rest with zero call-site changes.
Auth	PyJWT + PBKDF2	Algorithm-pinned, expiry-required JWT sessions; salted password hashing.
Console	React + Vite	Fast dev server with API proxy; small, dependency-light build.
Portal SPA	Vanilla JS	Zero build step; served same-origin by the portal service.
Directory	Microsoft Graph · ldap3	Cloud-native app-only auth; pure-Python LDAP with no native dependencies.
Encryption	Fernet (cryptography)	Authenticated symmetric encryption with a self-identifying token format, which makes the plaintext-to-ciphertext migration idempotent.
Charts	Hand-rolled SVG	Avoids a large charting dependency; full control of the instrument-panel look.

DB	SQLite / PostgreSQL	Zero-config locally; robust concurrent store in production.
Migrations	Alembic	Versioned, additive schema evolution.

4.4 Deployment topology

Locally, four processes run side by side: the measurement API (:8000), the Vite console (:5173, proxying `/api` to the backend), the remediation portal (:8100, serving its own SPA and calling the API over REST), and GoPhish (:3333, if present). A single Windows launcher provisions the Python 3.12 environment, installs dependencies and starts all of them. In production each service runs behind a reverse proxy that terminates TLS and caps request body size; environment variables set `ENV=production` (which disables docs, emits HSTS and makes ingestion fail closed), strong secrets including the Fernet encryption keys, locked CORS, rate limits, `DATABASE_URL` and `PUBLIC_BASE_URL`.

4.5 Request lifecycle

An authenticated console request carries its session in an httpOnly cookie; an API or script client carries a bearer JWT instead. A single dependency resolves whichever is present — the `Authorization` header takes precedence, and a cookie-authenticated state-changing request additionally requires a matching CSRF header (Section 12.4) — then decodes the token (pinned algorithm, required expiry), resolves the user and role, and enforces tenant scope before the handler runs. Ingestion requests are unauthenticated but secret-gated (HMAC signature for the webhook, shared token for the beacon), rate-limited, and fail closed in production. A recompute reads all of a tenant's events, runs the engine (applying any portal baseline floor) and writes cached score rows the dashboards then read. The portal calls the measurement API on a schedule with its scoped service token to read scores and, when its policy fires, writes back a floor or an in-person mark.

5. Data Model

5.1 Entity-relationship diagram

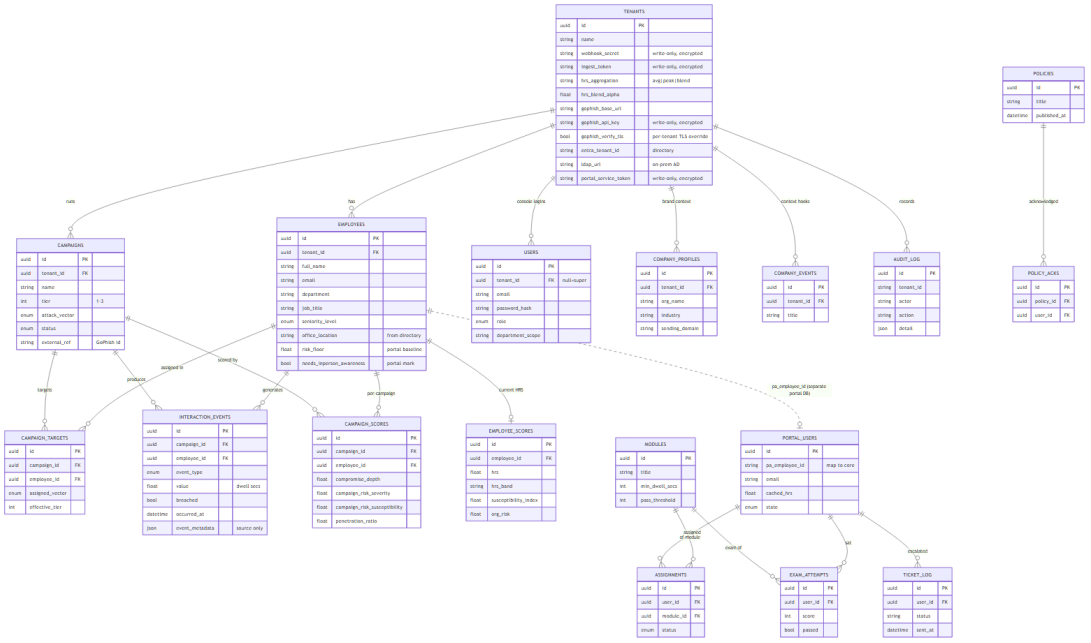


Figure 5.1 — Entity-relationship diagram, tenant-scoped tables. `InteractionEvent` is the scoring input; the score tables are a cached projection; no credential column exists anywhere. Fields marked *encrypted* are stored as Fernet ciphertext at rest.

5.2 Entity catalogue

- **Tenant** — organisation root. Holds the integration secrets (`webhook_secret` , `ingest_token` , `gophish_api_key` , `entra_client_secret` , `ldap_bind_password` , `portal_service_token`) — all write-only through the API *and* encrypted at rest — the GoPhish connection including its per-tenant `gophish_verify_tls` override, the directory connections, and per-tenant scoring overrides (`hrs_aggregation` , `hrs_blend_alpha`).
- **Employee** — roster entry: `full_name` , `email` (the join key for inbound events), `department` , `job_title` , optional `seniority_level` and `office_location` . It also carries the portal signals: `risk_floor` , and `needs_inperson_awareness` with `inperson_reason` / `inperson_since` .
- **Campaign** — `tier` (1–3), `attack_vector` , `status` , and `external_ref` (the GoPhish campaign id used to map inbound events and to link a console-launched campaign to scoring).
- **CampaignTarget** — per-recipient resolution of `assigned_vector` and `effective_tier` .
- **InteractionEvent** — one behavioural signal: `event_type` , `value` (dwell seconds), `breached` (only meaningful on `submitted` ; from an in-browser HIBP k-anonymity check [7] — the password never reaches the server), `occurred_at` , and `event_metadata` (provenance JSON, source only, never credentials).

- **CampaignScoreRow / EmployeeScoreRow** — cached engine output (compromise depth, severity and susceptibility risk, penetration ratio, HRS, band, Org-Risk).
- **User** — console login (email, PBKDF2 `password_hash`, role, optional `department_scope`). The application's own authentication, entirely separate from any simulated credential.
- **CompanyProfile / CompanyEvent** — organisational context; **AuditLog** — actor, action and a non-sensitive detail blob for side-effectful operations.

5.3 Tenant isolation

Every operational table carries an indexed `tenant_id` foreign key, and every query is filtered by it; a `super_admin` user has a null tenant and may cross tenants. Event ingestion additionally validates that a referenced campaign and employee belong to the caller's tenant, so a crafted import cannot attach events across tenants. On the portal signal surface, tenant scoping became *structural* when the service token replaced an operator login: the token *is* that tenant's token, so there is no tenant parameter left to get wrong.

5.4 Scores as a cache

`EmployeeScoreRow` and `CampaignScoreRow` are a materialised projection of `InteractionEvent`; they are never a source of truth. A recompute is idempotent and can rebuild all scores from events, which makes scoring reproducible and auditable and lets the aggregation model change without data migration.

5.5 The no-credential invariant

There is deliberately no column anywhere in the schema capable of holding a password or secret submitted by a target. Combined with ingestion-layer rejection (Section 12.2), this makes credential storage impossible by construction rather than by policy. Note the distinction from Section 12.3: the columns that *are* encrypted hold the organisation's own integration credentials — API keys and webhook secrets the operator configured — never anything a simulation target typed.

5.6 The portal's own schema

The remediation portal is a separate service with its own database — there are no shared tables. It holds `PortalUser` (employee accounts with a cached score snapshot), `Module` / `Question`, `Assignment` / `ExamAttempt` (progress with server-tracked dwell), `Policy` / `PolicyAck`, `TicketConfig` / `TicketLog`, and a `Setting` singleton (the PhishAwareness connection and escalation parameters). Its only link to the measurement core is the `pa_employee_id` mapping and the five REST signals in Section 10.3.

6. The Scoring Model

6.1 Overview

Scoring is a three-layer function evaluated per campaign and then aggregated per employee. All parameters live in `app/scoring/constants.py` so the model recalibrates without logic changes. The engine is pure: `score_campaign(events, context)` → `CampaignScore` and `score_employee(campaign_scores, seniority, as_of, aggregation, α)` → `EmployeeScore`.

6.2 Layer 1 — Compromise Depth (CD)

Within one campaign, each stage a target reaches contributes points; the total is capped at `CD_CAP = 100`.

Stage	Points	Stage	Points
opened	2	submitted	+30
clicked	15	download	+15
dwell > 10 s	+5	chatbot_turn	+8 (cap +30)
field_focus	+5	chatbot_disclosure	+15
typed_no_submit	+10	CD cap / campaign	100

```
CD = 0
for stage in reached_stages:      # each stage counted once
    CD += CD_POINTS[stage]        # chatbot_turn -> cap 30
CD = min(CD, 100)                 # CD_CAP
```

6.3 Reporting (protective behaviour)

Reporting a simulation is good behaviour and must move the score the right way. Reported **before** any click → `CD = 0` and a protective credit of −10 is applied to the campaign's severity risk. Reported **after** a click → accumulated depth is dampened: `CD ← CD × 0.4`.

6.4 Layer 2 — Tier difficulty coefficients

Coefficient	Tier 1	Tier 2	Tier 3	Drives
TIER_SEVERITY	1.0	1.3	1.6	Human Risk Score
TIER_SUSCEPTIBILITY	1.4	1.2	1.0	Susceptibility Index

Severity rises with tier because being compromised by the most realistic attack is the most dangerous outcome. Susceptibility falls with tier because being fooled by an easy lure that most people would catch is the strongest evidence a person is fool-able. The two views over the same events let a programme separate "dangerous when it happens" from "likely to happen".

6.5 Layer 3 — Decay, aggregation and banding

```
r_c = min(CD_c * tier_coeff_c, 100)         # campaign risk, capped
w_c = 0.5 ** (age_days_c / 90)              # 90-day half-life decay
```

```
average = sum(r_c*w_c) / max(sum(w_c), 1)    # evidence floor = 1
peak    = max(r_c*w_c)
HRS     = alpha*peak + (1-alpha)*average    # blend, alpha = 0.5
```

The `max(Σw, 1)` evidence floor prevents a weighted mean from cancelling the decay weight of a lone, stale campaign. The Susceptibility Index is computed identically but with `TIER_SUSCEPTIBILITY`. Bands: Low 0–24, Moderate 25–49, High 50–74, Critical 75–100. A per-employee `risk_floor` written by the portal raises the final HRS to at-or-above a baseline — never lowering a real score — so a freshly-onboarded hire reads Moderate until real behaviour supersedes it.

6.6 Penetration ratio and vectors

`penetration = depth_reached / vector_max_depth`, which normalises depth by how far the vector could take a target.

Vector	Reachable stages	Max CD
link_only	opened, clicked	17
credential_harvest	opened, clicked, dwell, field_focus, typed_no_submit, submitted	67
attachment_download	opened, clicked, download	32
chatbot	opened, clicked, chatbot_turn (≤30), chatbot_disclosure	62
mixed	resolved per-recipient to a concrete vector before scoring	—

6.7 Org-Risk overlay

`OrgRisk = HRS × role_sensitivity`, with Exec 2.0, Director 1.6, Manager 1.3, Staff 1.0. It is a prioritisation overlay, kept strictly separate from HRS so that an individual's own score is never inflated by their job title.

6.8 Worked examples

Scenario	CD	Tier 1	Tier 2	Tier 3
A — clicked only, severity	17	17.0 · Low	22.1 · Low	27.2 · Moderate
B — credential submit, severity	67	67.0 · High	87.1 · Critical	100 · Critical
B — credential submit, susceptibility	67	93.8 · Critical	80.4 · Critical	67.0 · High

The same submission is loudest on severity at Tier 3 (blast radius) but loudest on susceptibility at Tier 1 (foolability) — the inversion in action.

6.9 Calibration analysis

> The first implementation normalised campaign risk against a theoretical maximum of 160 (`CD_CAP` × highest tier). This compressed even a Tier-3 credential submission to ≈ 57 and pushed almost every employee into "Low". The model was recalibrated to cap `r` at 100 directly; a full compromise is now Critical on any tier and the bands recover their meaning. This is validated by the E1 gate (Section 16).

6.10 Determinism and complexity

Scoring is O(E) in the number of events plus O(C log C) per employee. Because the engine performs no I/O and reads a fixed parameter set, identical inputs and `as_of` always yield identical scores. This determinism is a deliberate design property rather than an implementation detail: it is what makes a score defensible to the person it describes, and it is the reason the model is arithmetic rather than learned (Section 1.5).

7. Ingestion and Integration

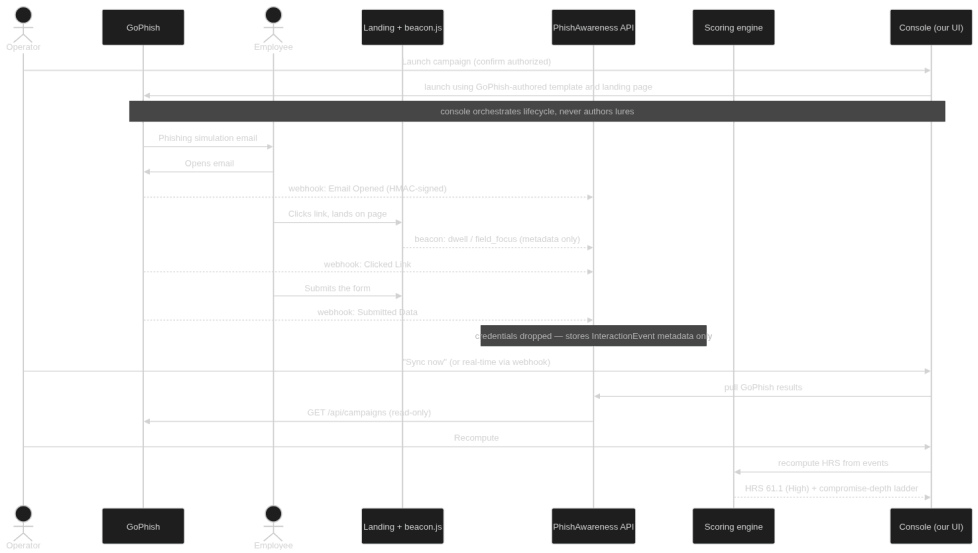


Figure 7.1 — Campaign sequence: launch, delivery, employee interaction, ingestion (submitted credentials dropped), sync/recompute, and score update.

7.1 GoPhish webhook (real-time, signed)

```
POST /api/v1/integrations/gophish/webhook?tenant_id=<uuid>
Header: X-Gophish-Signature: sha256=HMAC_SHA256(secret, body)

secret   = tenant.webhook_secret or env      # per-tenant first
verify   = hmac.compare_digest(expected, provided)   # constant-time
campaign <- external_ref == body.campaign_id
employee <- email == body.email
DROP     body.details.payload                # creds never stored
store    InteractionEvent(source="gophish", type=EVENT_MAP[msg])
```

`EVENT_MAP` translates GoPhish message names; non-scoring messages are acknowledged so GoPhish does not retry-storm. A `_safe_rid` helper extracts only the recipient id from `details`, discarding browser info and any submitted payload. In production the endpoint fails closed — an unsigned request is rejected when no secret is configured.

7.2 Landing-page beacon

The beacon captures depth signals GoPhish cannot see. `beacon.js` is safe by construction: it never reads an input's `.value`; the only scalar it transmits is dwell seconds, and every other signal is a content-free event.

```
POST /api/v1/integrations/beacon/events?tenant_id=<uuid>
token = tenant.ingest_token or env           # public-endpoint guard
_reject_credential_shapes(payload)           # 422 on cred-like keys
events: clicked, dwell(seconds), field_focus, typed_no_submit, submitted
```

Two richer signals — `download` and the chatbot events — are deliberately excluded from the beacon, since it is a public surface guarded only by a token embedded in a landing page. Those arrive instead through the authenticated ingestion endpoint from a caller's own instrumented surface.

7.3 Read-only GoPhish sync

```
campaigns = GET {gophish}/api/campaigns/     # read-only
for gc in campaigns:
    camp = link_or_create(external_ref=str(gc.id))
    for ev in gc.timeline:
        et = EVENT_MAP.get(ev.message); if not et: continue
        emp = get_or_create(Employee, email=ev.email)
        if not exists(camp.id, emp.id, et, time(ev)):   # idempotent
            add InteractionEvent(source="gophish", ingest="sync")
```

The dedup key makes a second sync a no-op. The read path never sends; the control path (Section 9) is what launches campaigns, under its own authorisation gate. TLS verification against the GoPhish admin API is a per-tenant setting, so one organisation's self-signed certificate cannot force every other tenant on a shared deployment to skip verification.

7.4 The event model and unmapped events

All inbound paths converge on a single `InteractionEvent` shape carrying `event_type`, an optional `value`, an optional `breached` boolean, `occurred_at` and provenance-only `event_metadata`. This uniform representation is what makes the scoring engine independent of how an event arrived.

When an inbound event references a campaign or recipient that cannot be resolved, the request is *accepted* (so a misconfigured sender does not retry-storm) but not stored, and the response carries only a generic reason code — `campaign_not_linked` or `employee_not_found`. Echoing the looked-up email back would turn a secret-gated endpoint into a roster-enumeration oracle. The identifiers an operator needs to fix the problem are instead recorded server-side: both paths log them, and the HMAC-verified webhook path additionally writes an `AuditLog` row visible in the operator's own console. The beacon path deliberately does not write that row, because its token is embedded in a public page and an audit write would let any reader of that page append unbounded rows to the tenant's audit trail.

8. Directory Integration (Entra ID & Active Directory)

The roster is the spine of the programme, so the platform imports it directly from the organisation's identity source. Two paths are offered from **Manage → Integration → Directory**, and both read directory metadata only — never anyone's password — and upsert employees idempotently by email (name, email, department, title, office, and a seniority band inferred from the job title, which feeds the Org-Risk overlay).

Entra ID (Microsoft Graph). Under an app registration with an application permission (`User.Read.All` or `Directory.Read.All`, admin-consented), the platform performs a client-credentials OAuth2 exchange against `login.microsoftonline.com`, then pages `GET https://graph.microsoft.com/v1.0/users` for the selected fields [2]. Pagination follows an `@odata.nextLink` that arrives in Graph's *own response body*; because that value is data rather than configuration, its scheme and host are validated against an allow-list before every outbound call, so a tampered or unexpected redirect target cannot turn pagination into an SSRF primitive.

On-prem Active Directory (LDAP). For organisations without cloud Entra, the same panel binds to a domain controller with a read-only service account (via the pure-Python `ldap3` library) and paged-searches for users under a configurable base DN and filter, mapping the standard AD attributes (`displayName`, `mail` / `userPrincipalName`, `department`, `title`, `physicalDeliveryOfficeName`). The URI is scheme-validated and the bind password is stored write-only and encrypted at rest.

Both connectors funnel through one shared upsert routine rather than duplicated per-connector logic, so matching and audit behaviour is identical regardless of which directory an organisation uses. Both flows are admin-gated to configure, operator-gated to run, and audit-logged.

9. GoPhish Campaign Control (operator surface)

Where the sync in Section 7.3 only reads, the console can also run the whole campaign lifecycle — while holding one line firmly: the deception artefacts stay authored inside GoPhish; the console selects and uses them but never composes or clones them.

- **Groups from roster.** Push a GoPhish target group built from the roster (everyone, a department, or a subset), idempotent by group name, so a directory re-sync keeps GoPhish in step.
- **Launch / schedule.** Pick an existing template, landing page and sending profile, a target group and a URL, then launch now or at a scheduled time. The console links the launched campaign to a local campaign by GoPhish id, so results flow straight into scoring.
- **Lifecycle.** Complete or delete a campaign from the console.

Guardrails. A launch requires an explicit "I confirm this simulation is authorized" confirmation (the API rejects a launch without it, HTTP 422) and is recorded in the audit log; every group push, launch, complete and delete is audited. If a chosen landing page has *Capture Passwords* enabled in GoPhish, the console shows a prominent warning — and, regardless, this platform never receives or stores those values. This keeps the console a full operator surface without turning the scorer into a lure factory.

10. The Awareness & Remediation Portal

Measuring risk is only half the job; the portal closes the loop. It is a separate service with its own database and its own employee logins, and it talks to the measurement core only over the REST contract below — so the core's boundary is never crossed.

10.1 Employee lifecycle

A portal user moves through four states. On sign-up the portal maps the person into PhishAwareness and assigns mandatory, non-skippable onboarding (an on-page lesson gated by a server-tracked dwell timer, followed by a passed exam). On completion it pushes a Moderate baseline (`risk_floor` ≈ 35) into scoring — an "unproven, treat as Moderate" floor that a real score later supersedes. Thereafter, a phishing failure (the synced HRS reaching High/Critical) or a newly published policy automatically assigns remediation along a ladder (standard → extended), and clearing it returns the person to good standing. If someone stays elevated past the configured tolerance, the portal marks them flagged: it sets the in-person-awareness mark in PhishAwareness and raises a ticket.

10.2 Training, exams and policies

Modules carry an authored lesson (kept in a small allow-listed HTML subset), an optional embedded video and a link to a real open-source resource [6], a minimum dwell, and an exam with a pass threshold. Non-skippable is enforced server-side: the exam will not accept answers until the recorded dwell meets the module's minimum, so completion cannot be forged from the UI. Admins publish policies (typed or uploaded documents) that employees must acknowledge, and publishing a policy can assign a remediation module to everyone.

10.3 The PhishAwareness REST contract

The portal authenticates to a dedicated `/portal` surface with a per-tenant service token carried in an `X-Portal-Token` header — generated and rotated from the console, encrypted at rest on both sides, and scoped to exactly these five endpoints.

Signal	Endpoint	Effect
read scores	`GET .../portal/employees`	pull each employee's HRS and band
resolve one	`GET .../portal/employees/by-email`	map by the email join key
ensure record	`POST .../portal/employees/ensure`	find-or-create on first sign-up
baseline	`POST .../portal/employees/{id}/risk-floor`	set/clear the Moderate floor
in-person mark	`POST .../portal/employees/{id}/inperson-flag`	raise/clear the mark

10.4 Ticketing integration (outbound)

When a person is flagged for in-person awareness, the portal POSTs a neutral JSON envelope to a webhook the organisation configures. It carries the employee, their risk, how long they have been elevated and a recommended action; the portal only sends the information and logs the attempt — what the ticket then does is the organisation's business. The webhook URL is scheme-validated, an optional auth header is supported (encrypted at rest), and a *Send test ticket* button verifies the wiring before any real escalation fires.

11. API Design

All `/tenants/{id}/...` routes are authenticated (cookie session or bearer JWT), enforce RBAC, and are tenant-scoped; only the two ingestion endpoints are public, and those are secret-gated. Interactive API docs are served in development and disabled in production. Request and response bodies are Pydantic models with enum and length validation; the event schema rejects credential-shaped keys so a mis-integrated sender cannot turn ingestion into a capture sink. The full endpoint catalogue is in Appendix E.

12. Security and Privacy Engineering

The platform was put through a dedicated secure-coding review, then a second hardening pass driven by a full-codebase audit. Both are recorded in `SECURITY.md`; this section summarises the threat model and the controls that resulted.

12.1 Threat model

Principal risks considered: (a) the platform being turned, by accident or intent, into a credential store; (b) forged or replayed ingestion; (c) cross-tenant data access; (d) session theft via XSS, cross-site request forgery, or broad browser-origin exposure; (e) brute-force against authentication; (f) running production on insecure defaults; (g) injection (SQL / LDAP / CSV-formula); (h) SSRF via operator-configured URLs; (i) stored XSS via admin-authored content; (j) memory exhaustion via large uploads; (k) disclosure of integration secrets from a database copy or backup; (l) over-privileged service-to-service credentials.

12.2 No credential surface (defence in depth)

Three independent layers make credential capture impossible: the schema has no column that can hold a submitted secret; ingestion drops `details.payload` and rejects credential-shaped fields; and the client `beacon.js` never reads an input's value. The only submission-related data stored are the boolean `submitted` and an out-of-band `breached` boolean from a client-side HIBP k-anonymity check that never transmits the password [7].

12.3 Encryption at rest for integration secrets

Integration secrets were originally *write-only* at the API boundary — exposed as a boolean status, returned at most once on rotation, never logged — but stored as plaintext columns. That protects them from an API consumer and not at all from anyone holding a database file or backup. All eight now live in Fernet-encrypted columns: six on `Tenant` (webhook secret, beacon ingest token, GoPhish API key, Entra client secret, LDAP bind password, portal service token) and two on the portal side (its PhishAwareness service token and the ticketing auth header).

The mechanism is a SQLAlchemy `TypeDecorator`, so encryption happens in the type layer and **no call site changed**: application code still assigns and reads plain strings. Migration is idempotent because every Fernet token carries a fixed prefix, which lets the decoder distinguish an already-encrypted value from legacy plaintext and pass the latter through on read rather than failing. Both services fail fast on startup if a production deployment is still using the development key.

12.4 Session handling: httpOnly cookies and double-submit CSRF

The console originally held its JWT in `localStorage` and sent it as a bearer header, which meant any XSS anywhere in the React application could read and exfiltrate a full operator session. The session now lives in an

httpOnly, Secure cookie that JavaScript cannot read, paired with a separate readable `csrf_token` cookie whose value must be echoed in an `X-CSRF-Token` header on every state-changing request — the double-submit pattern. Bearer authentication remains fully supported for API and script clients, and those requests skip the CSRF check, correctly: a cross-origin page cannot set a custom header without a CORS grant.

Two implementation decisions are worth defending. **CSRF is verified inside the authentication dependency, not as blanket middleware.** A middleware implementation cannot tell whether a given route uses cookie authentication at all, so it rejected public seed, webhook and beacon requests whenever a stale cookie happened to be attached — a fault that surfaced as broken tests but was equally a production defect. **Production cookies use `SameSite=None` with `Secure`, not `Lax`.** A `Lax` cookie is never attached to a cross-site fetch, which would have silently broken the documented split-origin deployment where the console and API sit on different hosts; this is safe because CSRF is enforced by the explicit token check rather than delegated to the cookie attribute.

12.5 Authentication and authorization

Console users authenticate with PBKDF2-hashed passwords (200k rounds, per-user salt, constant-time compare) and receive a JWT that is algorithm-pinned and expiry-required on decode. Four roles are enforced per endpoint and tenant-scoped; portal employees have their own native accounts, and portal admins are verified by delegating the login to PhishAwareness without persisting anything from it.

Capability	super_admin	org_admin	operator	dept_viewer
View dashboards	✓	✓	✓	own dept
Campaigns / roster · GoPhish control · directory sync	✓	✓	✓	—
Users / settings / secret rotation / delete campaign	✓	✓	—	—
Cross-tenant	✓	—	—	—

12.6 Least-privilege service-to-service authentication

The portal's background automation originally authenticated with a stored operator email and password. The defect was worse than the storage itself: the credential was re-written on *every* portal admin login, so the automation silently ran as whichever human signed in last, carrying that person's full blast radius — campaign launch, roster management, user administration. It now authenticates with a dedicated, rotatable, per-tenant token scoped to exactly the five `/portal` endpoints, so a leaked token reaches nothing else. Tenant scoping became structural in the process (Section 5.3). The portal's interactive admin login remains a separate, delegated check that persists nothing.

12.7 Observability of withheld detail

Hardening the unmapped-ingestion response to generic reason codes (Section 7.4) removed a roster-enumeration oracle, but initially left an operator unable to diagnose a misconfigured campaign at all, since the platform had no application logging of any kind. Logging was added for precisely this class of information: detail that is correctly withheld from a public response but must remain recoverable by someone with legitimate access. The asymmetry between the two ingestion paths — audit row for the HMAC-verified webhook, log only for the token-gated beacon — follows from how much the caller is trusted.

12.8 Hardening applied

- **Response headers and CSP.** Both services emit `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy` and a Content-Security-

Policy (`default-src 'none'` on the JSON API; a locked policy with `frame-ancestors 'none'` on the portal SPA); HSTS in production; `no-store` on API responses.

- **Injection and integrity.** CSV export neutralises spreadsheet formula injection; event ingestion rejects cross-tenant references and is per-row fault-tolerant; LDAP search uses an admin-set filter with no user-supplied interpolation; roster CSV import is validated through the same Pydantic schema as the manual endpoint, so length caps and enums apply on both paths.
- **Uploads.** Size-capped; portal policy files are extension-allow-listed with sanitised filenames (no path traversal).
- **XSS.** Admin-authored lesson and policy HTML is sanitised server-side through an allow-list parser; module resource and embed URLs are restricted to http(s).
- **SSRF.** Operator-configured outbound URLs are scheme-validated and remain admin/operator-gated and audited; Graph pagination targets are host-validated (Section 8).
- **Fail-closed in production.** The webhook and beacon reject unsigned or untokened requests; docs are disabled; both services fail fast on a default secret or default encryption key.
- **Rate limiting.** A per-IP fixed-window limiter guards the auth and ingestion endpoints.
- **Upgrade hygiene.** Credentials superseded by a design change are actively scrubbed on upgrade rather than left in place: a portal database created before the service-token change has its legacy operator email and password columns blanked at startup.

12.9 Residual and accepted risks

SSRF is inherent to "point the platform at your servers" features, mitigated by scheme and host checks, RBAC, audit, and an egress allow-list in a locked-down deployment. Admin-authored HTML is sanitised but ultimately admin-trusted. Stateless JWTs cannot be revoked before expiry; the TTL is short, and a denylist would be the next step if instant revocation became a requirement. The Fernet keys themselves live in environment variables and should move to a managed secret store in production. Dependency audits (`pip-audit` , `npm audit`) report no known vulnerabilities at the time of writing.

13. The Measurement/Delivery Boundary

A central design decision is that the platform measures what comes back and trains people, but never manufactures or sends deception. Bundling lure generation, page cloning, credential capture and sending into the same application would produce, in function, a credential-harvesting toolkit indistinguishable from a real phishing kit. By keeping delivery in GoPhish, restricting the measurement core to metadata scoring, orchestrating GoPhish without authoring its lures, and keeping the remediation portal strictly defensive, the project remains firmly within legitimate security engineering.

The separation is also an **audit-integrity** property, and this is the argument that most repays attention: the system of record for human risk never touches the sensitive act it measures. An organisation can hand the scoring platform's database to an auditor, a works council or a data-protection officer without exposing a single credential, message body or piece of lure content, because none of it was ever in scope to collect. A bundled product cannot make that claim. This is the same reasoning that led to declining the generative component in Section 1.5, applied at the level of system architecture rather than feature selection.

14. Implementation

- **Repository layout.** `backend/app/{api,scoring,integrations,db,synthetic,tests}`, `frontend/src/`, `portal/app/{...,static,tests}`, `backend/migrations/` (Alembic), `instrumentation/` (beacon), `docs/`.
- **Services and ports.** Measurement API :8000, console :5173, remediation portal :8100, GoPhish :3333. The portal reuses the backend's Python 3.12 environment and adds `ldap3`, `httpx` and `python-multipart`.
- **Migrations.** Additive, idempotent-per-column Alembic revisions: initial → integration and scoring columns → GoPhish sync → portal signals → Entra directory → LDAP directory → per-tenant GoPhish TLS → encrypted tenant secrets (which widens the columns before backfilling, since ciphertext is longer than the plaintext it replaces, and has a working downgrade) → portal service token. `create_all` is used by default for zero-config runs. The portal, having no Alembic, performs an idempotent startup schema patch instead.
- **Configuration.** Sensible zero-config defaults (SQLite, dev secrets, permissive dev CORS); production overrides `ENV` / `PORTAL_ENV`, strong secrets, the two Fernet encryption keys, `CORS_ORIGINS`, rate limits, upload caps, `DATABASE_URL`, `PUBLIC_BASE_URL`, the ingestion secrets and the GoPhish TLS setting.
- **Continuous integration.** Three GitHub Actions jobs run on every push and pull request: the backend suite plus the scoring-evaluation gate, the portal suite, and the frontend build.
- **Packaging.** A submission archive is produced with `git archive` via a helper script, so the export contains exactly the tracked tree (2.8 MB) rather than a working folder carrying virtual environments and dependency trees (≈ 111 MB).

15. Testing and Verification

Methodology. The scoring engine is exercised by scenario tests over pure functions; the API, integrations and portal are exercised in-process with FastAPI's `TestClient` over an in-memory SQLite `StaticPool`

(hermetic and fast). Network boundaries — Microsoft Graph, LDAP, the GoPhish REST API and the cross-service PhishAwareness client — are mocked so the real mapping, linking, escalation and guardrail logic is tested without live infrastructure. **119 automated tests pass (93 measurement core + 26 portal).**

Suite	#	Focus
test_scoring	17	CD, tier, decay, aggregation, bands, protective reporting
test_manage	11	campaign/roster/user CRUD, CSV import/export, RBAC
test_directory_gophish	11	Entra + LDAP mapping/sync, GoPhish pickers, group-from-roster, launch authorisation gate, complete/delete RBAC
test_security (core)	10	headers, JWT expiry/alg/key, cookie sessions, CSRF, SameSite by environment, CSV formula escaping
test_api	9	auth, ingestion, cross-tenant guard, dashboards
test_integrations	8	webhook HMAC, credential-drop, per-tenant secret, sync idempotency, unmapped-event auditing and the beacon's deliberate non-auditing
test_portal (core hooks)	7	floor raises-not-lowers, band recompute, in-person mark, RBAC
test_hardening	6	CORS lockdown, secret guard, rate-limit 429/off
test_crypto (core)	6	Fernet round-trip, idempotent encryption, legacy plaintext pass-through, migration backfill
test_eval	4	the E1 scoring-sanity gate
test_beacon_trends	4	beacon metadata, HRS-over-time
portal/test_portal_flow	9	onboarding→baseline, failure→remediation, policy→ack, stuck→in-person+ticket, dwell gate
portal/test_crypto	8	encryption round-trip, startup backfill, legacy credential scrub on upgrade
portal/test_pa_client	5	token-authenticated client, header propagation, failure handling
portal/test_security	4	HTML sanitiser, headers, webhook/URL scheme validation

Security regression. A dedicated suite locks in the hardening: headers, JWT expiry/algorithm/key enforcement, session-cookie attributes per environment, CSRF enforcement on cookie-authenticated mutations and its correct bypass for bearer clients, CSV formula-injection escaping, the HTML sanitiser, cross-tenant event rejection, outbound-URL scheme validation, secret encryption and the legacy-credential scrub. Static analysis (`bandit` , `ruff`) runs clean on the changed surface.

Live verification. Beyond automated tests, the running system was verified end-to-end. A real GoPhish open→click→submit chain flowed through the signed webhook and scored to HRS 61.1 ("High") with the credential payload dropped on arrival; a read-only sync discovered and ingested a campaign idempotently; rate-limiting and CORS lockdown behaved as specified. The cookie-session migration was verified in a real browser — confirming the session cookie is unreadable from JavaScript, that a state-changing request without the CSRF header is rejected, and that logout clears server-side state. Across both services, a live run confirmed the whole loop over real HTTP: a portal sign-up created the person in scoring, completing onboarding pushed the Moderate floor, and raising the person to Critical then syncing assigned remediation, set the in-person mark and fired a well-formed ticket (HTTP 200).

16. Evaluation

Three evaluation tracks address the claims that need evidence. E1 is fully automated and runs as a gate; E2 and E3 are human-run.

Track	Claim	Method	Status
E1 — Scoring validity	The score orders people the way a security team would expect	Nine scripted metamorphic scenarios asserting relationships any defensible model must satisfy	**9 / 9 pass**
E2 — Tracking accuracy	Recorded events match what recipients actually did	Scripted QA campaign against known actions; precision/recall/F1 per event type	Harness verified on synthetic logs; field run outstanding
E3 — Lure realism	Simulated lures elicit representative behaviour	Blind rater panel scoring realism 1–5 and guessing simulation vs. genuine	Aggregation verified on template data; panel outstanding

Why property-based testing for E1. No labelled ground-truth "true risk" exists for a person, so the score cannot be evaluated against a gold standard. Instead the evaluation asserts relationships that must hold for *any* defensible risk model: depth monotonicity (going further never lowers risk); tier severity ordering; the susceptibility inversion; submitted-Tier-1 outranking clicked-Tier-3, which is the headline judgement the model encodes; reporting as protective; recency decay; penetration fairness across vectors; Org-overlay separation from individual HRS; and correct behaviour at each band boundary. This also guards against regressions whenever the coefficients are re-tuned.

Live demonstration result. A real GoPhish campaign produced open→click→submit for a target; the platform scored the person at HRS 61.1 (High), compromise depth 47, penetration ≈ 0.70, with the submitted credential payload dropped on arrival. The remediation loop was demonstrated separately, end-to-end, across both services.

Threats to validity. E1 establishes internal validity — the model is self-consistent and behaves sensibly — not that its absolute numbers match real-world compromise probability; that calibration requires longitudinal outcome data. Open-tracking is inherently noisy: image proxies and security scanners inflate `opened` while blocked images suppress it, so E2's `opened` precision and recall are expected to be the weakest metrics and should be reported with that caveat. Click and submit events are far more reliable and carry far more weight in the score.

17. Results and Discussion

The platform meets its functional and non-functional requirements: it ingests via five paths, scores with an explicit and inspectable model, serves role-scoped analytics, populates the roster from the enterprise directory, runs the full GoPhish lifecycle from one console, closes the loop with automatic remediation and escalation, and enforces the no-credential invariant — all under a security-reviewed, 119-test codebase.

On the scoring model. Depth-aware scoring distinguishes a click from a submission; the inverted tier formulation separates severity from susceptibility, which is the most novel element of the design and the one that most clearly exceeds a click rate; 90-day decay lets an improving programme trend downward; the Org-Risk overlay supports prioritisation without penalising seniority; and the remediation portal turns a number into an action without ever crossing the measurement boundary. The calibration episode in Section 6.9 is itself a result worth stating: the original normalisation was defensible on paper and wrong in practice, and it was only visible

once real distributions were inspected — an argument for treating a scoring model as something to be validated empirically rather than derived and trusted.

On security as an iterative outcome. The security posture also demonstrates a process result. The first hardening pass addressed the conventional web-application surface. A subsequent full-codebase audit found three issues that a checklist-driven review had not: secrets that were write-only at the API but plaintext at rest; a session model safe against network attackers but not against XSS; and a service credential that was over-privileged in a way no single file revealed — it was only visible by tracing when the credential was written. A later independent comparison against the pre-hardening tree found a fourth class again: a diagnosability regression introduced *by* a security fix, where withholding an identifier from an attacker also withheld it from the operator. The lesson is that security fixes have second-order effects on operability, and that adversarial review at more than one level of abstraction finds different defects.

18. Limitations

- Results to date rest on synthetic data plus controlled live demonstrations; field validation (E2/E3 and an authorised live campaign against real staff) is outstanding, and no claim is made about calibration against real compromise rates.
- The default SQLite store and synchronous recompute suit a demonstration; at scale, PostgreSQL and a background recompute/sync job are needed.
- The portal is single-org per instance, and its non-skippable dwell is time-based rather than attention-based; a real SCORM/xAPI player [5] would strengthen proof-of-viewing.
- The Fernet keys protecting the secret columns live in environment variables; a managed secret store is the correct production answer.
- The interface is English-first; Arabic/RTL is specified but not implemented.
- The platform contains no machine-learning component, so the "AI-powered" designation describes the project's original framing and roadmap rather than the delivered implementation (Section 1.5).

19. Future Work

Complete E2/E3 and one authorised live campaign; move the encryption keys to a secret manager and add an egress allow-list; background recompute and sync with horizontal scaling; a SCORM/xAPI training player; multi-org portal hosting; and Arabic/RTL localisation.

Where machine learning genuinely belongs. Two applications would extend the platform without reopening the boundary argument of Section 1.5, because neither generates deception. **Adaptive difficulty selection** — choosing which tier and vector to send a given person next, given their history — is a sequential decision problem over data the platform already holds, and would make the programme itself adaptive rather than the lures. **Narrative explanation** — turning a score and its depth ladder into readable guidance for a manager or the employee — is a text-generation task over non-sensitive metadata, and would address the usability gap between an explainable number and an actionable one. Both keep the deterministic score as the system of record, with any model strictly downstream of it, so the auditability property established in Section 13 is preserved.

20. Conclusion

The project delivers a complete, tested, security-reviewed phishing human-risk platform: a transparent measurement core, enterprise directory sync, full single-console GoPhish control, and a separate remediation portal that closes the loop from score to training to escalation — all built around a principled boundary that

keeps deception generation out of the scorer and credentials out of the database. That boundary also accounts for the most visible difference from the Capstone 1 proposal: the generative component was declined on safety grounds, and the resulting architecture is stronger for it, because the system of record for human risk never touches the act it measures.

The engineering result is a working MVP demonstrated end-to-end across both services, covered by 119 automated tests and hardened over two review passes plus an independent audit. The research result is a scoring model that is explicit enough to defend line by line: every number a person is measured by can be traced to an event, a coefficient and a decay term. Field validation and at-scale hardening are the identified path to production.

References

[1] GoPhish — Open-Source Phishing Framework (MIT License). REST API and webhook reference. *https://getgophish.com*

[2] Microsoft. Microsoft Graph REST API (`/users`) and Entra ID application permissions. *https://learn.microsoft.com/graph*

[3] OWASP. Application Security Verification Standard (ASVS); social-engineering testing and CSV-injection guidance.

[4] NIST. SP 800-50, *Building an Information Technology Security Awareness and Training Program*; SP 800-53, *Security and Privacy Controls for Information Systems and Organizations*.

[5] ADL / IEEE. SCORM and xAPI e-learning interoperability standards.

[6] CISA, UK NCSC, and SANS *OUCH!* — free and openly reusable security-awareness resources referenced by the training library.

[7] Have I Been Pwned. Pwned Passwords k-anonymity range API (client-side breached-password check; the password is never transmitted).

[8] Kosinski, M. (2026). *What is phishing?* IBM Think. https://www.ibm.com/think/topics/phishing

[9] Anti-Phishing Working Group. *Phishing Activity Trends Report, Q1 2025*. https://docs.apwg.org/reports/apwg_trends_report_q1_2025.pdf

[10] BARR Advisory (2025). *Top Takeaways from the 2025 IBM Cost of a Data Breach Report*. https://www.barradvisory.com/resource/takeaways-data-breach-report/

[11] Altwaijry, N., Al-Turaiki, I., Alotaibi, R., & Alakeel, F. (2024). Advancing Phishing Email Detection: A Comparative Study of Deep Learning Models. *Sensors*, 24(7).

[12] Ovi, M., Rahman, M., & Hossain, M. (2024). PhishGuard: A Multi-Layered Ensemble Model for Optimal Phishing Website Detection. *arXiv preprint*.

[13] Vulfin, A., Sulavko, A., Vasiliev, V., & Minko, A. (2026). A Multimodal Phishing Website Detection System Using Explainable Artificial Intelligence Technologies. *Machine Learning and Knowledge Extraction*, 8.

[14] Xue, Y., Spero, E., Koh, Y., & Russello, G. (2025). MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*.

[15] *Context-Aware Spear Phishing: Generative AI-Enabled Attacks Against Individuals via Public Social Media Data* (2025). arXiv preprint. https://arxiv.org/pdf/2605.11268

[16] Gelman, H., Hastings, J., & Kenley, D. (2025). An Ethically Grounded LLM-Based Approach to Insider Threat Synthesis and Detection. *arXiv preprint*.

[17] Sharma, G., Thakur, A., & Tiwari, C. (2024). Developing a Comprehensive Framework for User and Entity Behavior Analytics (UEBA). *Journal of Communication Engineering & Systems*, 14(2).

[18] Alzaabi, F., & Mehmood, A. (2024). A Review of Recent Advances, Challenges, and Opportunities in Malicious Insider Threat Detection Using Machine Learning Methods. *IEEE Access*, 12.

[19] Okika, N., Okoh, O., & Etuk, E. (2025). Mitigating Insider Threats and Social Engineering Tactics in Advanced Persistent Threat Operations through Behavioural Analytics. *International Journal of Advance Research Publication and Reviews*, 2(3).

[20] Arctic Wolf (2024). *Solving Security Awareness Program Challenges*. https://arcticwolf.com/resources/blog/5-biggest-security-awareness-challenges/

[21] Hoxhunt (2024). *Human Risk Management Playbook*. https://hoxhunt.com/guide/human-risk-management-playbook

[22] KnowBe4. *Security Awareness Training and Simulated Phishing Platform*.

[23] Proofpoint. *Cybersecurity and Compliance Solutions*.

[24] Hoxhunt. *Phishing Training Product Overview*.

Appendix A — Lure Difficulty Tiers

Tier	Sophistication	Personalisation	Typical tells	Severity	Susceptibility
1	Low / bulk	None (generic greeting)	Obvious: wrong sender, errors, implausible pretext	1.0	1.4
2	Moderate / semi-targeted	Light (name, department)	Subtle: look-alike domain, routine pretext	1.3	1.2
3	High / spear	Heavy (role, project, manager)	Few or none; requires process, not eye	1.6	1.0

Appendix B — Running the Platform

Double-click `Start-PhishAwareness.bat` : it provisions a Python 3.12 virtual environment, installs dependencies, and starts the API (:8000), the console (:5173), the remediation portal (:8100) and GoPhish (:3333, if present), opening the console and the portal. In the console, seed a demo organisation and sign in; in the portal, an admin signs in with their PhishAwareness operator login and employees create their own accounts.

One additional step wires the portal's *background automation*: in the console, under **Manage → Integration → Awareness & Remediation Portal**, generate the service token and paste it into the portal's Settings screen. This is deliberately manual — the token is minted per tenant against a running API, and the admin login no longer stores a credential on the portal's behalf (Section 12.6). Training and the console work without it; only the risk-floor and in-person-mark sync require it.

Configuration defaults to SQLite with no environment variables; the production checklist (secrets, encryption keys, CORS, rate limits, TLS, `ENV=production`) is in `SECURITY.md` . Tests: `python -m pytest app/tests -q` from `backend/` and from `portal/` . A submission archive is produced with `.\make-submission.ps1` .

Appendix C — Selected Constants

`CD_POINTS` (opened 2, clicked 15, dwell +5, field_focus +5, typed_no_submit +10, submitted +30, download +15, chatbot_turn +8, chatbot_disclosure +15); `CD_CAP` 100; `CHATBOT_TURN_CAP` 30; `REPORTED_AFTER_CLICK_MULTIPLIER` 0.4; `PROTECTIVE_CREDIT_R` −10; `TIER_SEVERITY` {1.0, 1.3, 1.6}; `TIER_SUSCEPTIBILITY` {1.4, 1.2, 1.0}; `DECAY_HALF_LIFE_DAYS` 90; `CAMPAIGN_RISK_CAP` 100; `EVIDENCE_FLOOR` 1; bands {Low 0–24, Moderate 25–49, High 50–74, Critical 75–100}; `ROLE_SENSITIVITY` {Exec 2.0, Director 1.6, Manager 1.3, Staff 1.0}; portal `onboarding_floor` 35, `inperson_after_days` 30.

Appendix D — Security Controls at a Glance

Auth: PBKDF2-200k with constant-time compare; JWT HS256, algorithm-pinned, expiry-required; httpOnly Secure session cookie with double-submit CSRF for the console, bearer tokens for API clients. **AuthZ:** four roles, per-tenant and per-department scope; a scoped, rotatable service token for cross-service automation. **Secrets:** eight integration secret columns encrypted at rest with Fernet; write-only at the API; production fail-fast on default keys; legacy credentials scrubbed on upgrade. **Ingestion:** HMAC-signed webhook and token-gated

beacon, metadata only, credential-shape rejection, fail-closed in production, cross-tenant reference guard. **Web:** security headers and CSP, HSTS in production, docs disabled in production, CORS locked. **Integrity:** CSV-formula escaping, capped and sanitised uploads, server-side HTML sanitiser, http(s)/ldap(s) scheme checks, Graph host allow-list. **Ops:** per-IP rate limiting, audit log with real actor attribution, application logging for withheld diagnostics, clean `pip-audit` and `npm audit`. Full detail in `SECURITY.md`.

Appendix E — API Surface

Group	Representative endpoints	Auth
Auth	`POST /api/v1/auth/login` (sets session cookies, returns a bearer token) · `POST /auth/logout`	public
Dashboard	`GET /tenants/{t}/dashboard/{summary, departments, campaigns, employees, employees/{id}, trend}`	session/JWT + role
Manage	`.../manage/{campaigns, employees, users}` · `.../employees/import` · `.../export/employees.csv`	operator/admin
Integration — GoPhish	`.../integration/gophish/{config, status, campaigns, sync, templates, pages, smtp, groups}` · `.../groups/from-roster` · `.../campaigns/launch` · `.../campaigns/{id}/complete` · `DELETE .../campaigns/{id}`	operator/admin
Integration — Directory	`.../integration/{entra, ldap}/{config, status, preview, sync}`	admin (config) / operator
Portal signals	`.../portal/employees` · `.../employees/by-email` · `.../employees/ensure` · `.../employees/{id}/{risk-floor, inperson-flag}`	`X-Portal-Token`
Ingestion (public)	`POST .../integrations/gophish/webhook` · `POST .../integrations/beacon/events`	HMAC / token